

BACM: Transparent, Provenance-Aware Shared Memory for Parallel LLM Agents on Changing Codebases

Tejashvi Kumawat

Abstract

When several large language model (LLM) agents work on the same codebase in parallel, each agent re-reads the same files and re-derives the same facts, and once the code changes, every agent keeps relying on what it read before. Shared memory removes the redundancy but makes the second problem worse: a stale fact in shared memory misleads every agent that reuses it. We present **BACM** (Branch-Aware Context Manager), a shared context manager for parallel agents that stores reusable knowledge as typed objects with provenance (file, version and supporting span), dependency edges and derivation rules. When a source changes, BACM invalidates exactly the affected part of the dependency graph and recomputes only the part that some open question needs. A first design that exposed this memory to the model as tools was *more* expensive than independent agents, because every memory operation is a model turn that re-sends the whole context. BACM therefore makes memory *transparent*: the tool layer publishes what agents read, serves already-read files as short digests, and answers questions from known facts by typed rules, so the model never calls the memory directly. We evaluate BACM on REPOQA-DYN, a benchmark built from five real Python repositories, in which a real release lands in the middle of a run and ground truth comes from static analysis. In a pre-registered evaluation on held-out seeds with GPT-5 mini, BACM answered 89% of the questions correctly on static repositories and 88% when the repository changed mid-run, against 58% and 49% for independent agents with context eviction (both differences significant after Holm correction), and reduced stale answers after the release from 61 to 7 of 186. Token efficiency is workload-dependent and not established statistically: pooled token use changed by -22% and -26% (not significant), from large savings on two repositories to large increases on the others. The central finding is that reasoning quality is not state consistency: agents that reason more read the old code more accurately but are the most often stale after a release, whereas BACM, which tracks provenance and dependencies, is not. We also report the limits of the approach: on static repositories, independent agents that reason more are as accurate and cheaper, and facts extracted by the model propagate their errors to every agent that reuses them. Two defects in our own implementation, found after the pre-registered runs, are disclosed with frozen and corrected results side by side. Code, prompts, cached model responses and analysis scripts are released at <https://github.com/tejashvi-kumawat/bacm>.

Keywords: multi-agent systems, large language models, agent memory, provenance, incremental computation, cache invalidation, software repositories

1 Introduction

Running several LLM agents in parallel is an increasingly common way to tackle large tasks: agents explore independently, the wall-clock time falls, and the diversity of their trajectories improves coverage [11, 23, 24]. Parallelism also wastes work. Agents that investigate the same repository open the same files, issue the same searches and reconstruct the same facts, and because an agent re-sends its whole context at every turn, each redundant read is paid for again on every later turn.

Sharing a memory between agents is the natural remedy [4, 19], but it raises a problem that recent work identifies as the central open challenge of multi-agent memory: *consistency* [26]. Codebases change while agents work on them. A shared statement such as “class `Leaf` has one method” may silently become false after a commit, and every agent that reuses it inherits the error. Techniques that make inference cheaper, such as KV-cache sharing [15, 29] or prompt compression [13], reduce the price of each token but are blind to whether the reused content is still *true*.

This paper asks a narrow, testable question: *can a shared context manager reduce the tokens that parallel agents spend while keeping their answers correct when the source code changes during the run?* We answer it with a system, BACM, a benchmark built on real repositories and real releases, and a pre-registered evaluation. We organise the evaluation around four research questions:

- RQ1 Accuracy and cost.** Does a shared, provenance-aware memory improve answer accuracy, and at what token cost, compared with independent agents and with a shared memory that does not track staleness?
- RQ2 Staleness.** When a real release lands in the middle of a run, how many final answers are stale, that is, still equal to the pre-release truth?
- RQ3 Mechanism.** Which components of the system carry the effect?
- RQ4 Robustness.** Do the gains persist against a stronger baseline whose agents reason more, and with a second model family?

A database view of agent memory. BACM treats reusable knowledge the way incremental view maintenance treats derived data [7]. Every base fact carries provenance [6] down to the span of code that supports it; every derived fact carries dependency edges and a typed derivation rule. A source change therefore invalidates only the facts whose supporting spans changed, together with everything derived from them, and only the part of this affected subgraph that some open question needs is recomputed. The design follows truth maintenance systems [1], demand-driven incremental computation [8] and build systems [20]; what is new is that the extractor that turns source code into facts is itself a fallible language model.

The interface matters as much as the data structure. Our first implementation exposed this memory as tools that the model calls (`kb_query`, `kb_publish`). It kept answers current, but it was *more* expensive than independent agents, because every memory operation is an additional model turn that re-sends the entire context (Eq. (3)). BACM is instead *transparent* to the model (Fig. 1): agents only list, search, read and answer, and a mediator in the tool layer publishes what they read, serves files that another agent has already read as compact digests, answers questions deterministically from known facts, tells agents which facts are still missing, and maintains answers after a change without waking the agents.

Contributions.

1. A formal model of shared agent knowledge with span-level provenance, dependency edges and typed derivation rules, with a soundness property for reuse, a bound on the work of incremental recomputation, and a token cost model, validated on measured runs, that explains why memory exposed as tools backfires (Sections 3 and 4).
2. BACM, a transparent context manager that combines focused on-demand extraction, rule-based question resolution, demand-driven incremental recomputation and small-context re-derivation (Section 5).
3. REPOQA-DYN, a benchmark on five real Python repositories in which a real release lands in the middle of a run, with exact ground truth from static analysis (Section 6).
4. A pre-registered evaluation on held-out seeds with paired significance tests, ablations, a reasoning control, a second model family and a symbol-index upper bound, reported together with its negative results and two post-registration corrections (Sections 7 and 8).

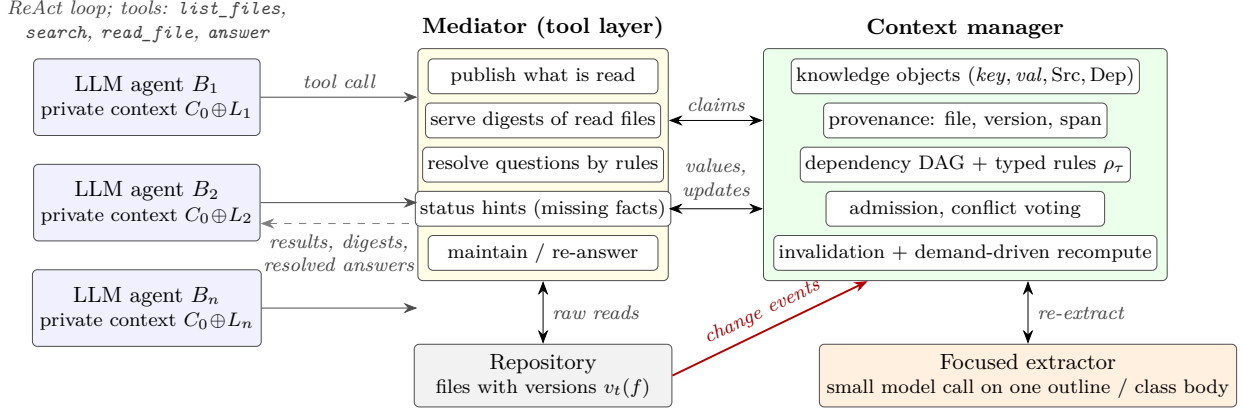


Figure 1: Architecture of BACM. The agents (left) never call the memory. The mediator (centre) sits in the tool layer: it turns every file an agent reads into knowledge objects with provenance, serves already-read files as digests, resolves open questions from known facts by typed rules and attaches the list of still-missing facts to each tool result. The context manager (right) stores the objects in a dependency DAG. When the repository changes (bottom), it invalidates the affected subgraph and recomputes the demanded part with focused, small-context extraction calls.

Summary of findings. The main result is about consistency, not cost. BACM substantially and significantly improves accuracy on static and on changing repositories, and after a release it leaves only a small fraction of the stale answers that the baselines produce. A second, more general observation is that *reasoning quality is not state consistency*: independent agents that reason more are as accurate as BACM on static repositories, yet they are the most often stale after a release, because reasoning more about what was read does not reveal that it has since changed. Token efficiency is workload-dependent: the average token use falls, but the saving is not statistically significant and turns into an increase on some repositories. We therefore claim consistency under change, and we do not claim lower cost.

2 Related Work

Memory for LLM agents. Agent memory has been organised as hierarchical context management [21], retrieval over an experience stream [22], retrieval augmentation over external corpora [17], and a rapidly growing set of designs surveyed by Zhang et al. [28] and Hu et al. [12]. These mostly concern a single agent and do not track whether stored knowledge is still true of a changing environment.

Sharing between parallel agents. Multi-agent frameworks coordinate agents through conversation [24], standard operating procedures [11], role play [23] or debate [2]; state is shared by passing messages. Learning to Share [4] adds a shared memory bank with a learned admission controller so parallel teams reuse intermediate results, and reports lower runtime at matched task performance. DeLM [19] replaces a central coordinator with a shared context and a task queue to which agents write verified updates asynchronously. Both reduce redundant work; neither versions knowledge or invalidates it when the underlying sources change. Yu et al. [26] argue from a computer-architecture perspective that multi-agent memory consistency is the most pressing open challenge, by analogy with consistency models for multiprocessors [16], and Han et al. [9] list shared-state management among the open problems of multi-agent systems. Our work targets exactly this gap, and adds an interface argument: how memory is exposed to the model largely decides whether sharing saves

tokens.

Reducing inference cost. Serving systems share attention state across requests with common prefixes [15, 29], and prompt compression shortens contexts [13]. These are complementary to our work: they make each token cheaper, while we decide which tokens need to be sent at all and whether what is reused is still valid.

Incremental computation and provenance. Maintaining derived data under change is classical: truth maintenance systems keep justifications and retract dependent beliefs [1]; incremental view maintenance recomputes only affected tuples [7]; provenance semirings formalise why a derived tuple holds [6]; demand-driven incremental computation recomputes only what is requested [8]; and build systems rebuild only stale targets [20]. We transfer these ideas to agent memory, where the extractor that turns sources into facts is itself a fallible model.

Positioning. None of the individual ingredients of BACM is new: provenance, dependency graphs, cache invalidation and incremental recomputation are established, and shared memory for agents exists. The contribution lies in their combination for a setting where all of them are needed at once: parallel LLM agents sharing derived facts about an environment that changes during the run. Two aspects distinguish this setting from classical incremental computation. First, the component that turns sources into facts is itself an imperfect language model, so sharing makes extraction errors correlated and the manager must keep each fact’s supporting span tight enough to be both checkable and sufficient. Second, the interface through which the model reaches the memory determines whether sharing saves or costs tokens, a question that classical systems do not face.

Agents on code. Repository-level benchmarks such as SWE-bench [14] evaluate end-to-end task success, and repository-level retrieval improves code completion [27]. We instead need questions with exact, recomputable answers under controlled source changes, which we obtain from static analysis of real repositories at two releases. Long contexts are used unevenly by models [18], one more reason to keep each agent’s context small. Agents follow the ReAct pattern [25].

3 Problem Formulation

We first fix the setting and the vocabulary used in the rest of the paper; Table 1 summarises the notation.

Table 1: Notation.

Symbol	Meaning
$B_1, \dots, B_n$	parallel agents (branches)
$E_t, \text{text}_t(f), v_t(f)$	environment at time t ; text and version of file f
$C_0, C_i(t) = C_0 \oplus L_i(t)$	shared initial context; private context of branch B_i
Q_i	open questions of branch B_i
$k \in \mathcal{K}$	knowledge object (a fact) in shared memory
$\text{Src}(k), (f, v, s)$	provenance: file, version and supporting span
$\text{Dep}(k), G$	parents of a derived fact; the dependency DAG
ρ_τ	typed derivation rule for keys of type τ
$s \sqsubseteq \text{text}$	span s occurs in the text as a complete unit
$A(\Delta), D$	facts affected by a change Δ ; facts demanded by open questions
N, c_0, g	turns of a branch; initial context; context growth per turn
ε	error rate of the extractor

Environment and branches. A team of n agents (branches) $B_1, \dots, B_n$ works on a shared environment whose state at time t is a set of files $E_t = \{(f, \text{text}_t(f), v_t(f))\}$ with monotone version counters $v_t(f)$. Every branch starts from the same initial context C_0 (task description and its questions Q_i) and keeps a private context $C_i(t) = C_0 \oplus L_i(t)$, where $L_i(t)$ is its own log of turns; private contexts are never synchronised.

Definition 1 (Knowledge object). *Shared knowledge is a set $\mathcal{K}$ of objects*

$$k = (\text{key}_k, \text{val}_k, \text{Src}(k), \text{Dep}(k), \text{conf}_k, \text{status}_k), \quad (1)$$

where $\text{Src}(k) \subseteq \{(f, v, s)\}$ records the file, version and supporting span s (the text the value was read from), and $\text{Dep}(k) \subseteq \mathcal{K}$ the objects k was derived from.

Base facts have $\text{Dep}(k) = \emptyset$; derived facts have $\text{Src}(k) = \emptyset$ and are computed by a typed rule $\text{val}_k = \rho_{\tau(k)}((\text{val}_d)_{d \in \text{Dep}(k)})$ for the key type $\tau(k)$. $G = (\mathcal{K}, E)$ with $E = \{(d, k) : d \in \text{Dep}(k)\}$ is a DAG. For example, the rule

$$\text{total}(c) = \text{n}(c) + \text{total}(\text{base}(c))$$

for `method_total` has three parents: the class’s own method count `n_methods`, its base class `base_of`, and the base’s total (Fig. 4).

Definition 2 (Validity). *A knowledge object is valid at time t iff*

$$\begin{aligned} \text{valid}_t(k) \iff & \forall (f, v, s) \in \text{Src}(k) : s \sqsubseteq \text{text}_t(f) \\ & \wedge \forall d \in \text{Dep}(k) : \text{valid}_t(d), \end{aligned} \quad (2)$$

where $s \sqsubseteq \text{text}$ means that s occurs in the text as a complete unit: a line, or a block that is not extended by further lines at its indentation.

File-level provenance replaces the span condition by $v_t(f) = v$; span-level provenance tolerates edits elsewhere in the file. The “complete unit” condition matters for spans that are blocks: a class body to which a method was appended still *contains* the old body as a substring, but no longer supports the old method count (Section 8.2).

Definition 3 (Span-sound extraction). *An extractor χ maps a key and a view of a file to a value. It is span-sound if the value of a base fact is a function of its supporting span: $s \sqsubseteq \text{text}_t(f) \Rightarrow \chi(\text{key}, \text{text}_t(f)) = \chi(\text{key}, s)$.*

A symbol index is span-sound by construction; a language model is span-sound only up to an error rate ε , which we measure (Section 8). The span must cover *everything* the value depends on: the header line of a class supports its base class, but its method count needs the whole body.

Objective. Each branch must answer its questions Q_i with values that equal the ground truth on the environment *at the end of the run*, while the team spends as few model tokens as possible.

4 Theoretical Analysis

Proposition 1 (Soundness of reuse). *If the extractor is span-sound and every rule ρ_τ is a function of its parents’ values, then every object served as valid at time t has the value that re-extracting and re-deriving it from E_t would produce.*

Proof sketch. Induction over a topological order of G . For a base fact, validity (2) means its span is present in the current text, so span-soundness gives the same value. For a derived fact, all parents are valid and by the induction hypothesis carry current values; ρ_τ is a function, so the derived value is current. $\square$

The two propositions are an adaptation of well-known results on provenance and incremental view maintenance to agent memory, not new theory; their value here is that they state precisely which assumption each defect of our own implementation violated (Section 8.3). Proposition 1 is why the manager can reuse knowledge and *maintain answers* without asking any agent: an answer that is a derived object stays correct for as long as it is valid, and becomes invalid exactly when one of its supporting spans changes.

Proposition 2 (Incremental work). *Let a change touch the spans Δ . Let $A(\Delta)$ be the base facts whose spans are in Δ together with all their descendants in G , and D the facts some open question depends on (its demanded closure). Demand-driven incremental recomputation performs $|A(\Delta) \cap D|$ focused extractions or rule applications, while global recomputation performs $\Theta(|K|)$ and, with agents, re-runs whole branches.*

Proof sketch. Objects outside $A(\Delta)$ keep their spans and valid parents, so by Eq. (2) they remain valid and need no work. Objects in $A(\Delta) \setminus D$ are marked stale but not recomputed until demanded. Each object in $A(\Delta) \cap D$ is recomputed once, in topological order, after its parents. $\square$

The same bound holds per object for the token cost when each extraction reads only the supporting view of the fact (one class body or the file outline) instead of the whole file.

A token cost model for tool-using agents. A branch that runs N turns re-sends its whole context every turn. With initial context c_0 and average growth g tokens per turn, its input tokens are

$$T(N) = \sum_{j=0}^{N-1} (c_0 + gj) = Nc_0 + \frac{g}{2}N(N-1). \quad (3)$$

Measured on our held-out runs, Eq. (3) explains most of the variation in a branch’s input tokens (Fig. 2). Three consequences shape the design. **(i) Interface turns are expensive.** Exposing memory as tools the model must call adds m turns, each costing the *current* context: $\Delta T \approx m(c_0 + gN)$. Our first design paid many such turns per run and was more expensive than independent agents (Section 9). Transparent memory sets $m = 0$. **(ii) Lower growth compounds.** Serving a file another branch already read as a digest of $d \ll |f|$ tokens reduces g for every later turn, a quadratic effect in Eq. (3). **(iii) Resolved questions remove turns.** A question answered by rules from known facts removes the turns the agent would have spent deriving it, and those are the most expensive turns because they come late.

Error propagation. Sharing makes errors correlated. If a stored fact is wrong with probability ε and r branches reuse it, all r inherit the error, whereas independent branches err (more) independently. For a question that needs q facts, the probability that all are right is $(1 - \varepsilon)^q$ in both regimes, but the shared regime concentrates errors. Lowering ε at the source therefore matters more with sharing than without, which motivates *focused extraction*: each fact is extracted from its smallest supporting view, on demand, with a little reasoning spent where the context is small.

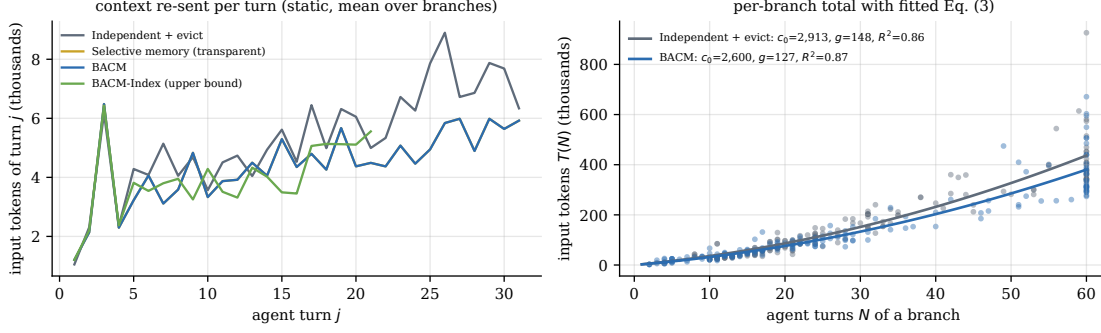


Figure 2: The cost model against measurement (GPT-5 mini, held-out runs, replayed from the response cache). Left: input tokens of the j -th agent turn, i.e. the context re-sent at that turn. Right: total input tokens of each branch against its number of turns, with the least-squares fit of Eq. (3). Fitted on 360 held-out branches (static and dynamic): Independent + evict $c_0 = 2,913$, $g = 148$, $R^2 = 0.86$; BACM $c_0 = 2,600$, $g = 127$, $R^2 = 0.87$.

5 The BACM System

BACM consists of two components (Fig. 1). The *mediator* wraps the tools that agents already use and is the only place where agents and memory meet; the *context manager* stores knowledge objects, maintains the dependency graph and reacts to source changes. Each agent is an ordinary ReAct loop [25] with four tools, `list_files`, `search`, `read_file` and `answer`; it is never told about the memory’s internals and never calls it. Algorithm 1 gives the mediator’s hook around a tool call, Algorithm 2 the reaction to a change, Fig. 4 a worked example, and Fig. 5 the course of a complete run.

Algorithm 1 Mediator hook around every tool call of agent B_i

Require: tool call $(op, args)$ of branch B_i ; open questions Q_i

- 1: **if** $op = \text{read_file}(f)$ **and** a digest of f at version $v_t(f)$ exists **then**
 - 2: **return** digest restricted to facts in the demanded closure of Q_i ▷ lowers growth g
 - 3: $r \leftarrow \text{execute}(op, args)$ on the repository
 - 4: **if** $op = \text{read_file}(f)$ **then**
 - 5: $F \leftarrow \text{OUTLINEFACTS}(f)$ ▷ one small model call on top-level lines
 - 6: **for all** $(key, val, s) \in F$ **do**
 - 7: $\text{PUBLISH}(key, val, (f, v_t(f), s))$
 - 8: **for all** count facts demanded by some open question and not yet known **do**
 - 9: extract from the body of that one class; PUBLISH with the whole body as span
 - 10: **for all** branches B_j , open questions $q \in Q_j$ **do**
 - 11: **if** q evaluates from valid facts by rules ρ_τ **then**
 - 12: record answer q as a derived object; tell B_j it need not act
 - 13: **return** $r + \text{resolved answers} + \text{missing facts of } Q_i$ (status hint)
-

5.1 Publishing what agents read

When an agent reads a file, the mediator converts it into base facts with provenance (f, v, s) before returning the text. With *model-based* extraction, which is the configuration we evaluate as BACM, a small model call reads only the file’s *outline*, that is, its top-level `class` and `def` lines, and lists structural facts such as which file defines a class and what its base class is. Each fact is stored with

Algorithm 2 Change handling: invalidation and demand-driven recomputation

Require: changed file f at new version v'

```
1:  $R \leftarrow \emptyset$ 
2: for all valid objects  $k$  with a source  $(f, v, s)$  do
3:   if span  $s$  is still present (as a complete block) in  $text(f)$  then
4:     replace the source by  $(f, v', s)$  ▷ re-validated, no model call
5:   else
6:      $R \leftarrow R \cup \{k\}$ 
7:    $A \leftarrow \text{MARKSTALE}(R)$  ▷  $R$  and all descendants in  $G$ , BFS order
8:    $D \leftarrow \bigcup_j \text{closure}(Q_j)$  under the latest known values ▷ demand
9:   for all  $k \in A \cap D$  in topological order do
10:    if  $k$  is a base fact then
11:      re-extract  $k$  from its focused view of the new text
12:    else
13:      re-derive  $k$  by  $\rho_{\tau(k)}$  from its (recomputed) parents
14:    notify consumers only if the value changed; derived answers update in place
15: answers that are not derived objects: re-answer in a small fresh context, else wake the agent
```

its supporting line as span. Counting facts are expensive to extract and only some of them are needed, so they are extracted *on demand*: when an open question needs the number of methods of a class, the manager extracts it from the body of that one class and stores the whole body as span. We call this *focused extraction*: every fact is read from the smallest view that supports it. Because these contexts are small, the manager’s extraction calls can afford a higher reasoning effort than the agents’ turns. The *BACM-Index* configuration replaces the model by a symbol index built with static analysis; it serves as an upper bound for a perfect extractor.

5.2 Serving reads from memory

When an agent reads a file that has already been read and whose version has not changed, the mediator returns a *digest* instead of the raw text: the facts of that file that are relevant to the reader’s open questions, that is, those in the demanded closure of the questions under the current values. The raw text remains available on request. Digests lower the per-turn context growth g in Eq. (3), and the saving recurs at every later turn of the reading agent.

5.3 Resolving questions without the model

After every tool call, the manager tries to evaluate every open question of every branch from valid facts by applying the typed rules ρ_{τ} recursively. When this succeeds, the answer is recorded, the agent is informed that it need not work on that question any further, and the answer is stored as a derived object that depends on its premises, so that a later change reaches it. When it fails, the agent receives the list of facts that are still missing, which acts as a deterministic query plan. Answers that an agent produces itself are checked against verified facts; if the facts disagree, the verified value is recorded and the agent is told why.

5.4 Admission and conflicts

A claim enters shared memory only if its admission score $S(k) = w_1R + w_2C + w_3V + w_4D - w_5U$ exceeds a threshold θ , where the terms measure relevance, confidence, strength of verification, downstream demand and uncertainty, and only if its source version is current and its dependencies

are valid. Contradictory claims are kept together with their provenance and resolved by a majority of independent observations, adding verification reads when necessary. Figure 3 shows the resulting life cycle of a knowledge object.

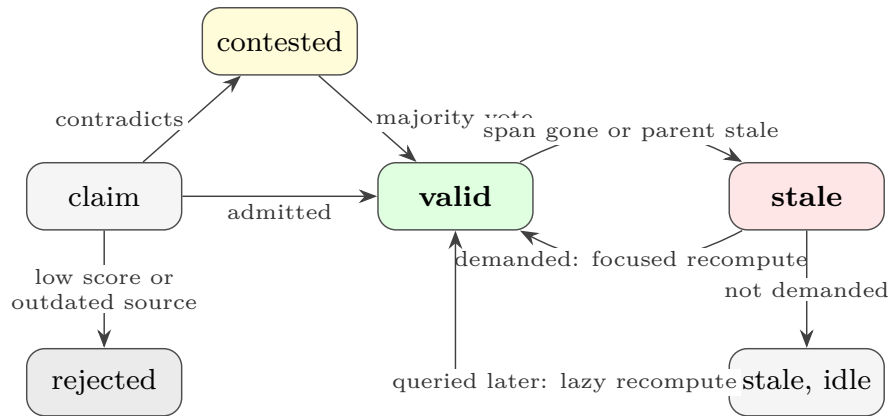


Figure 3: Life cycle of a knowledge object. A claim is admitted, contested or rejected. A valid object becomes stale when its supporting span disappears or a parent becomes stale; it is recomputed at once if an open question demands it and otherwise only when it is next queried. A change that leaves an object’s span intact merely updates its version, without any model call.

5.5 Change handling

When a file changes, the manager checks every valid object that has a source in that file (Algorithm 2). If the supporting span still occurs in the new text as a complete unit, the object is re-validated by updating its version. Otherwise it becomes stale, and staleness propagates along the dependency edges (Proposition 1). Demand-driven recomputation then rebuilds, in topological order, only the stale objects in the demanded closure of some open question (Proposition 2): base facts are re-extracted from their focused views of the new text, and derived facts are re-derived by their typed rules once their parents have been rebuilt. Consumers are notified only when a value actually changes. Because an answer resolved by rules is itself a derived object, it is updated in place and the agent is not woken (Fig. 4). Answers that the manager could not derive itself, for instance because a chain leaves the repository, are re-derived in a small fresh model context from the current facts; the agent is woken only if that is impossible.

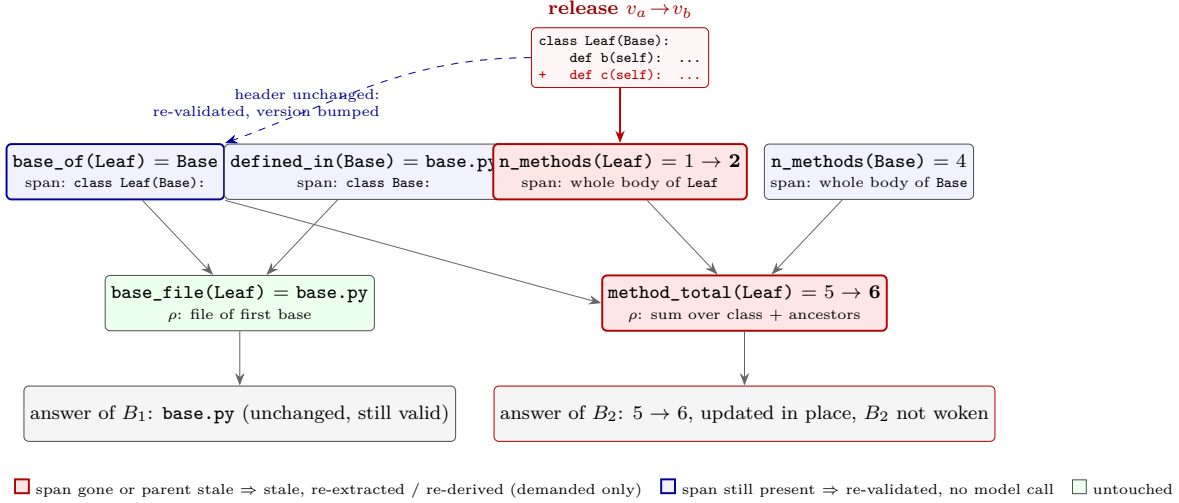


Figure 4: Worked example of change handling. A release appends `def c` to class `Leaf`. The header span of `base_of(Leaf)` is still present, so this fact is re-validated without a model call (blue). The body span of `n_methods(Leaf)` no longer occurs as a complete block, so the count and everything derived from it become stale (red). Because an open question demands them, the count is re-extracted from the new class body and the total is re-derived by its rule; the answer that depends on it is updated in place. `base_file(Leaf)` does not depend on the changed span and is untouched (green).

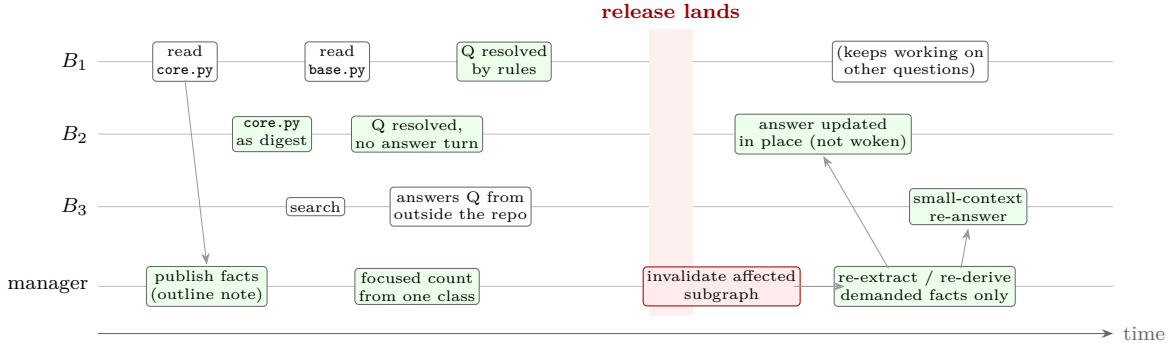


Figure 5: Course of a run with a mid-run release, schematically. Before the release, each file is published once and served to other agents as a digest, and questions are resolved by rules without answer turns. After the release, the manager invalidates the affected subgraph and recomputes only the demanded facts; answers are maintained in place, and an answer that cannot be derived by rules is re-derived in a small context, all without waking the agents.

6 The RepoQA-Dyn Benchmark

Repositories and changes. Five real Python projects pinned at a release, each with a later release used as the mid-run change (Table 2). In dynamic runs, once half of all questions have been answered, the working tree moves to the later release, as after a `git pull`: every changed source file changes at once. Answers are scored against the code after the change.

Questions. Five kinds, sampled with equal weight about classes whose base class is defined in the repository (Table 3). Each run has 3 agents with 6 questions each, drawn from a shared pool, so information needs overlap. In dynamic runs half of the questions of each kind concern facts that change between the releases.

Table 2: Repositories. Classes = classes whose base class is defined in the repository (question subjects); changed = share of those classes with at least one answer that changes between releases.

Repository	Releases	Source files	Tokens (approx.)	Classes	Changed
httpx	0.18.0 → 0.27.0	21	61,882	50	30%
requests	v2.25.0 → v2.32.0	18	43,584	28	100%
flask	2.0.0 → 3.0.0	21	75,302	13	23%
click	7.1 → 8.1.0	17	69,125	22	73%
rich	v10.0.0 → v13.0.0	73	179,885	61	16%

Table 3: Question kinds. The last column gives the base facts an answer needs, where k is the number of in-repository ancestors of the class.

Kind	Asks for	Base facts needed
base_file	the file that defines the class’s first base class	2
grandbase	the base class of that base class	2
family_files	the files of the class and of all its in-repository ancestors	$1 + 2k$
n_methods	the number of def statements directly inside the class body	1
method_total	n_methods summed over the class and its in-repository ancestors	$1 + 2k$

Ground truth. Answers are computed with Python’s **ast** module on the files as they are at answer time. Names defined in more than one file, and classes whose base has the class’s own name (an external class it shadows), are excluded. Before running any model we checked that the tool layer’s parsing agrees with the **ast** ground truth on every base fact and that every derived answer equals its rule applied to its parents (Appendix B).

Why this design. Questions require chaining several facts across files, counting questions require reading a class body carefully, and the release changes are real refactors (files moved, classes re-parented, methods added). In requests, for example, the package moved under **src/** between the two releases, so every file-valued answer changes; this is why all of its classes count as changed in Table 2. Ground truth is exact and recomputable, so accuracy is decided by the models’ reading, not by a judge.

7 Experimental Setup

Systems compared. All systems use the same agents, tools, prompts and models and differ only in what the agents share; Table 4 summarises them.

Table 4: Systems compared. “Shared” states what agents share; “Staleness” whether stored knowledge is versioned, invalidated and recomputed after a change.

System	Shared	Staleness	Role
Independent + evict	nothing	–	strong baseline: raw file text evicted after notes
Independent + evict, low effort	nothing	–	reasoning control: every agent call at higher effort
Selective memory (transparent)	key–value facts	no	isolates the staleness machinery
BACM	knowledge objects	yes	the proposed system (model extraction)
BACM-Index	knowledge objects	yes	upper bound (symbol-index extraction)

Independent + evict gives each agent a private context only; after an agent has noted the facts it needs from a file, the raw text is evicted from its history. This is a strong context-hygiene base-

line, considerably cheaper than keeping raw text. The *low-effort* variant runs every agent call at a higher reasoning effort and controls for the possibility that BACM’s gains come merely from the reasoning spent in its extraction calls. The *selective memory* uses BACM’s transparent interface and extraction with a gated key–value store in the spirit of Fiorese et al. [4], but without provenance, versions, dependencies or invalidation; comparing it with BACM isolates the contribution of staleness tracking. A plain independent baseline and a selective memory accessed through tools were part of the pre-registered plan but were dropped, by a logged amendment and before any held-out result was inspected, when the measured cost per run turned out twice as high as estimated; both appear in our earlier benchmark (Table 11).

Models. Agents use GPT-5 mini at its lowest reasoning effort (“minimal”); the manager’s small-context extraction and re-derivation calls use “low”. A replication uses Gemini 3.5 Flash-Lite on the free tier. Every model response is cached, so every analysis in this paper can be replayed without further API calls.

Protocol. All design decisions were made on development seeds. The held-out evaluation uses seeds 200–205, which were fixed in advance together with the hypotheses, systems and analysis in a pre-registration that records a fingerprint of the frozen code; every later change is logged as an amendment (Appendix A). Each system runs on 5 repositories $\times$ 6 seeds in a static and in a dynamic environment, giving 30 runs per cell. The ablations use the first three held-out seeds (15 runs per cell).

Metrics. *Question accuracy* is the share of a run’s questions whose final answer equals the ground truth at the end of the run. *Tokens* are the input plus output tokens of every model call in the run, including agent turns, extraction, manager calls and re-derivations. *Stale answers* are, among the answers whose ground truth changed during the run, those whose final value equals the pre-change truth.

Statistics. We report means with 95% bootstrap confidence intervals [3]. Systems are compared on paired runs that share repository, seed and environment, using two-sided sign-flip permutation tests [5] with Holm correction [10] over all comparisons. Token changes are pooled, that is, total over total, so that inexpensive runs do not dominate the ratio.

8 Results

We report the results by research question. Table 5 gives the main held-out numbers for every system; Table 6 gives the paired significance tests on which all claims of significance rest. Unless stated otherwise, all numbers refer to GPT-5 mini on the 30 held-out runs per cell.

Table 5: Held-out evaluation (GPT-5 mini). Mean and 95% bootstrap CI over 30 runs (5 repositories $\times$ 6 held-out seeds). Tokens include every model call (agents, extraction, manager).

Architecture	Static repository		Repository changes mid-run	
	Accuracy	Tokens (k)	Accuracy	Tokens (k)
Independent + evict	0.58 [0.51, 0.65]	561 [433, 698]	0.49 [0.43, 0.55]	592 [452, 757]
Independent + evict, all agents low effort	0.93 [0.91, 0.96]	353 [281, 449]	0.63 [0.57, 0.68]	387 [297, 514]
Selective memory (transparent)	0.89 [0.85, 0.92]	435 [344, 525]	0.64 [0.59, 0.70]	504 [389, 621]
BACM	0.89 [0.85, 0.92]	435 [344, 525]	0.88 [0.85, 0.91]	438 [343, 534]
BACM-Index (upper bound)	0.95 [0.91, 0.97]	314 [224, 412]	0.93 [0.91, 0.96]	369 [277, 465]

8.1 RQ1: Accuracy and token cost

Each cell of Table 5 averages 30 held-out runs (5 repositories, 6 seeds). On static repositories BACM answered 0.89 of the questions correctly, against 0.58 for independent agents with eviction (+0.31, $p_{\text{Holm}} = 0.0006$). When the repository changed mid-run the figures were 0.88 and 0.49 (+0.39, $p_{\text{Holm}} = 0.0006$). BACM used 22% fewer tokens on static and 26% fewer on changing repositories, but neither difference is significant ($p_{\text{Holm}} = 0.99$ and $p_{\text{Holm}} = 1$). Per repository, the change on changing repositories was click -65% , flask -57% , httpx $+69\%$, requests $+55\%$, rich $+18\%$; on static ones it was click -61% , flask -62% , httpx $+44\%$, requests $+170\%$, rich $+18\%$. On static repositories BACM and the selective memory produce identical runs, as they should: provenance, versions and invalidation only act when a source changes. On changing repositories the staleness machinery is worth $+0.24$ accuracy over the selective memory ($p_{\text{Holm}} = 0.0006$) at 13% fewer tokens (not significant).

Accuracy against cost. Figure 6 places every system in the accuracy–token plane, with 95% confidence intervals on both axes; a better system lies further up and further to the left. On static repositories (left panel), the systems that share facts form a cluster well above independent agents with eviction, at similar or lower cost, and the low-effort control reaches the same accuracy level by reasoning more. When the repository changes (right panel), the picture separates: BACM and BACM-Index lose little accuracy, whereas every system without staleness tracking loses more: the selective memory and the low-effort control fall by about 0.25–0.30, and independent agents with eviction fall from an already low level. The per-run distributions in Fig. 7 show that the differences are not driven by a few outlying runs: the interquartile ranges of BACM and of independent agents with eviction do not overlap in either environment.

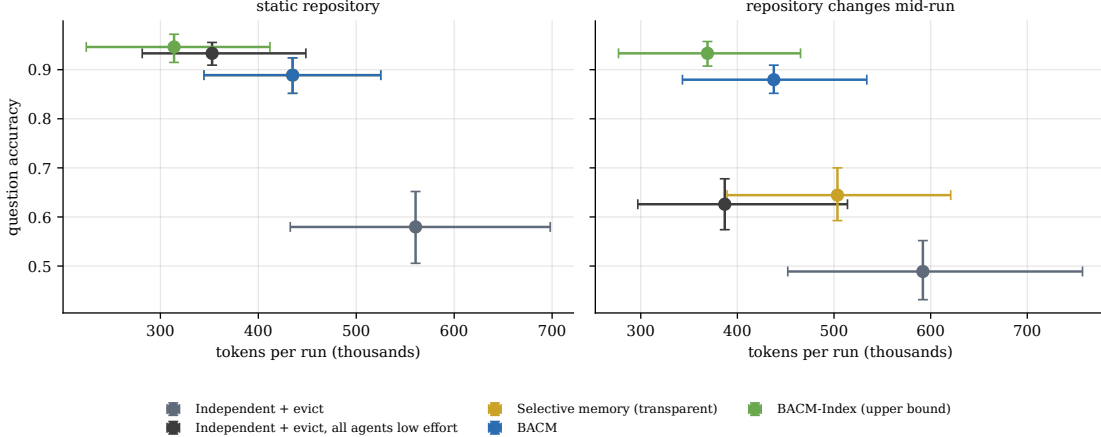


Figure 6: Accuracy against tokens per run on the held-out runs (means with 95% bootstrap confidence intervals over 30 runs per system). Better systems lie up and to the left. Left: static repositories. Right: a real release lands in the middle of each run.

Why the token saving is not significant. The per-repository view in Fig. 8 explains the large confidence interval of the token change. Its left panel shows that BACM’s accuracy advantage holds on every repository. Its right panel shows the token change of BACM relative to independent agents with eviction: on click and flask, where agents otherwise re-read large files many times, sharing removes more than half of the tokens; on httpx, requests and rich, whose relevant classes sit in fewer and smaller files, BACM spends more than the baseline, because the extraction calls and the longer tool results cost more than the redundant reads they replace. Whether sharing saves

Table 6: Paired comparisons of BACM against each baseline: same repository, seed and environment. Two-sided sign-flip permutation tests, Holm-corrected over all rows. Token change is pooled (total over total).

Env.	Baseline	Δ accuracy	p_{Holm}	Δ tokens	p_{Holm}
static	Independent + evict	+0.309	0.0006	-22%	0.986
static	Independent + evict, all agents low effort	-0.044	0.262	+23%	0.0852
static	Selective memory (transparent)	+0.000	1	+0%	1
dynamic	Independent + evict	+0.391	0.0006	-26%	1
dynamic	Independent + evict, all agents low effort	+0.254	0.0006	+13%	0.262
dynamic	Selective memory (transparent)	+0.235	0.0006	-13%	1

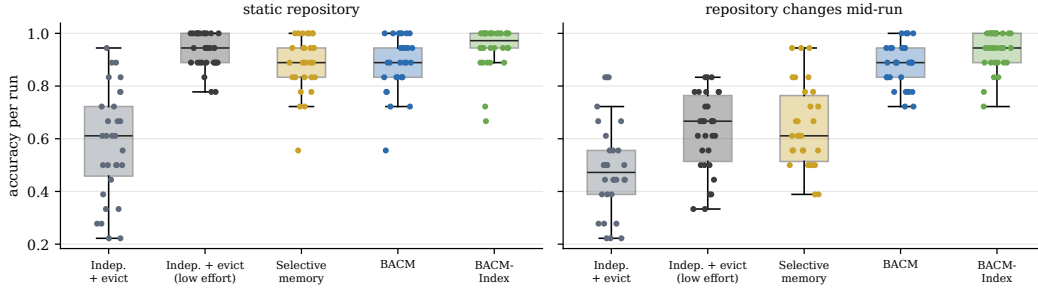


Figure 7: Accuracy of every held-out run (dots) with quartiles (boxes). Each dot is one run of one repository and seed; 30 dots per box.

tokens is therefore a property of the task’s redundancy, not a constant of the method.

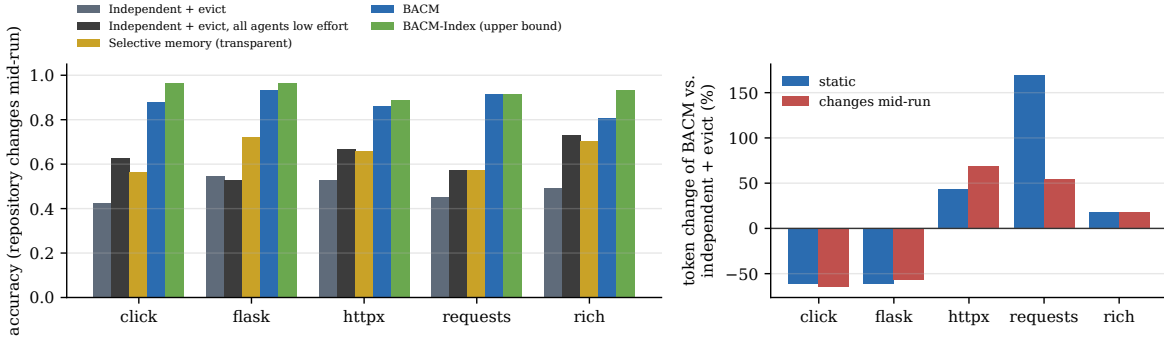


Figure 8: Per-repository results. Left: accuracy of every system when the repository changes mid-run (means over 6 seeds). Right: pooled token change of BACM relative to independent agents with eviction, on static and on changing repositories; negative values are savings.

Where the tokens go. Figure 9 decomposes each system’s tokens by the caller that spends them. Compared with independent agents, on static repositories BACM’s agent turns cost 66k tokens less per run, independent agents spend 70k on fact notes that BACM does not need, and BACM’s manager spends 10k on extraction and re-derivation (2% of its total); on changing repositories BACM’s agent turns cost 84k tokens less per run, independent agents spend 92k on fact notes that BACM does not need, and BACM’s manager spends 22k on extraction and re-derivation (5% of its total). The manager’s calls are cheap because each of them reads one outline or one class body rather than a whole file.

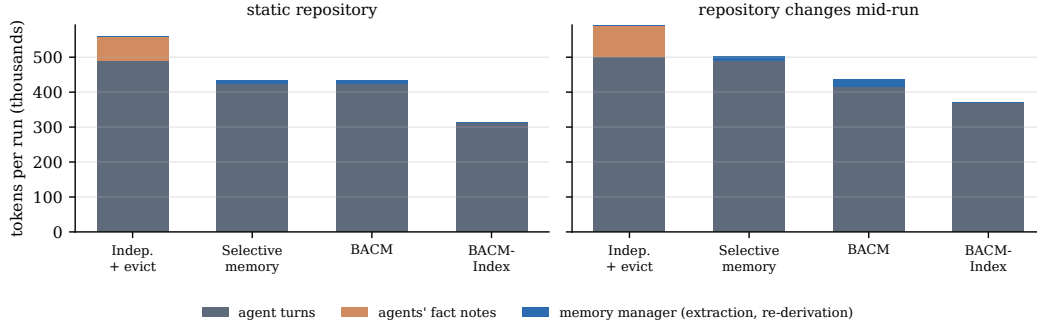


Figure 9: Tokens per run by caller (held-out means, replayed from the response cache). “Agents’ fact notes” exist only for independent agents, which must note the facts of a file before its raw text is evicted; “memory manager” covers BACM’s extraction and re-derivation calls.

Accuracy by question kind. Figure 10 breaks accuracy down by the five question kinds of Table 3. On static repositories BACM answers the structural questions almost perfectly (`base_file` 1.00, `grandbase` 0.99, `family_files` 1.00) and single method counts (1.00, against 0.51 for independent agents). The hardest kind is `method_total`, which sums model-extracted counts over a chain of classes: 0.66 for BACM on static repositories and 0.65 after a release, against 0.90 and 0.92 with the symbol index. One wrong count anywhere in the chain makes the total wrong, and with shared memory the same wrong count reaches every agent (Section 4).

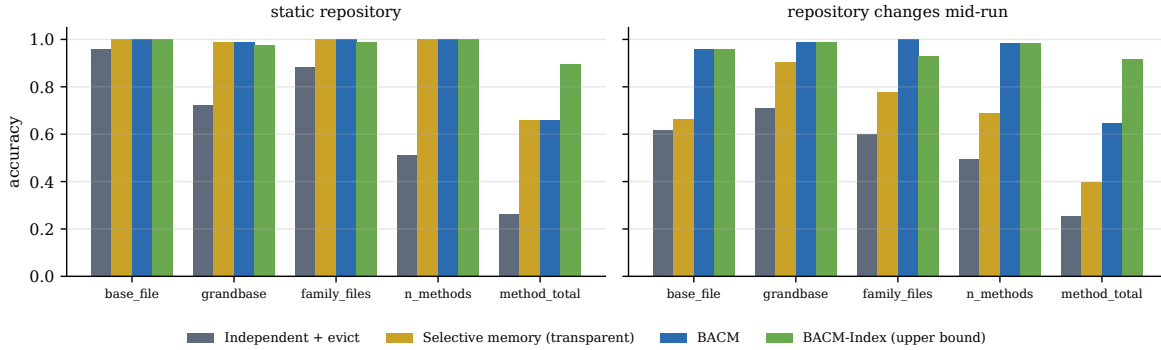


Figure 10: Accuracy by question kind on the held-out runs (left: static; right: a release lands mid-run). Structural kinds chain facts across files; `n_methods` and `method_total` require counting, the latter over a chain of classes.

8.2 RQ2: Stale answers after a release

Of the 186 final answers whose ground truth changed during a run, 7 were stale with BACM, against 61 for independent agents with eviction, 131 for the selective memory and 167 for independent agents that reason more. The last number is the surprising one: agents that reason more read the pre-release code more accurately, and then keep what they read. Figure 11 breaks down every final answer whose ground truth changed during the run into three outcomes: current (correct), stale (equal to the pre-release truth) and otherwise wrong. Independent agents keep what they read before the release, and the more carefully they read, the more often their final answer is the old truth. The selective memory has no notion of version and keeps serving pre-release facts to every

agent. BACM invalidates and recomputes the affected facts and is current for 171 of these 186 answers; the remaining 15 errors divide almost evenly between stale and otherwise wrong answers, and most of them involve model-extracted counts.

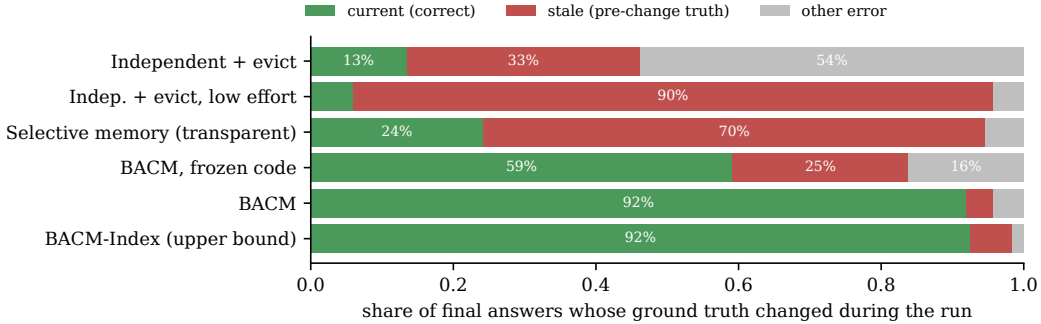


Figure 11: Outcome of every final answer whose ground truth changed during the run, pooled over the 30 held-out dynamic runs per system: current (correct), stale (equal to the pre-change truth) or otherwise wrong. “BACM, frozen code” is the pre-registered implementation before the corrections of Section 8.3.

8.3 Post-registration corrections

Analysing the held-out results, we found two defects in our own implementation. Both were fixed, covered by regression tests, logged in the pre-registration, and followed by a re-run of the two affected systems on the same held-out seeds; the baselines have no context manager and are unaffected. Table 7 reports all three versions, and all other tables and figures report the final one.

Fix 1 (provenance span). The span recorded for a method count covered only the class header line, so a release that changed a class body without touching its header left the count marked valid. Proposition 1 assumes that the span covers everything the value depends on, and this one did not. The span is now the whole class body, accepted only as a complete block (Eq. (2)).

Fix 2 (recursive rebuild). When a release made a *derived* parent stale, for example the method total of an ancestor class, recomputation rebuilt only stale base parents and gave up on the dependent answer, which was then re-derived from incomplete facts. Stale parents are now rebuilt recursively. In the same change, recomputed derived facts are re-derived by their typed rule, as everywhere else in the system, rather than by a model call.

All 60 static re-runs of each version reproduce the frozen results exactly, since the corrected code paths are only exercised after a change. Because the corrections were found by inspecting held-out results, the corrected numbers should be read as the performance of the intended design rather than as a fully pre-registered result; the frozen numbers in Table 7 are the strictly pre-registered ones.

8.4 RQ3: Which components matter

Table 8 and Fig. 12 remove one component at a time. Accuracy drops most without automatic question resolution (static -0.29 , changing -0.24) and without reasoning in manager calls (static -0.19 , changing -0.09). Digest-served reads did not pay off on static repositories: without them accuracy changed by $+0.02$ and tokens by -6% , while on changing repositories removing them cost -0.03 accuracy and $+9\%$ tokens. Resolving questions by rules carries most of the effect on both accuracy and tokens: without it, agents derive the chains themselves, which costs additional turns

Table 7: BACM on the frozen code and after each post-hoc fix (same held-out seeds; baselines are unaffected). Fix 1 extends the provenance span of a method count to the whole class body; fix 2 rebuilds stale derived parents recursively and re-derives derived facts by their typed rule (Section 8.3). Stale = stale answers / answers whose truth changed, pooled over 30 runs.

Env.	System	Frozen code			Fix 1			Fix 2 (reported)		
		Acc.	Tok. (k)	Stale	Acc.	Tok. (k)	Stale	Acc.	Tok. (k)	Stale
static	BACM	0.89	435	–	0.89	435	–	0.89	435	–
static	BACM-Index (upper bound)	0.95	314	–	0.95	314	–	0.95	314	–
dynamic	BACM	0.76	449	46/186	0.79	468	13/186	0.88	438	7/186
dynamic	BACM-Index (upper bound)	0.91	365	14/186	0.93	375	11/186	0.93	369	11/186

and introduces chaining and arithmetic errors. Spending a little reasoning on the manager’s small-context extraction calls is the second most important choice. Invalidation and answer maintenance matter only when the repository changes, as expected: on static repositories, removing them has no effect at all. The ablations were run on the pre-registered code and are compared with the pre-registered reference.

Table 8: Ablations of BACM: change when one component is removed (paired runs; positive token change = more expensive without it). Run on the pre-registered (pre-fix) code against the pre-fix reference.

Env.	Removed component	Δ accuracy	Δ tokens	pairs
static	reasoning in manager calls	-0.189	+15%	15
static	digest-served reads	+0.019	-6%	15
static	eager invalidation	+0.000	+0%	15
static	answer maintenance	+0.000	+0%	15
static	automatic question resolution	-0.293	+74%	15
dynamic	reasoning in manager calls	-0.093	+18%	15
dynamic	digest-served reads	-0.026	+9%	15
dynamic	eager invalidation	-0.078	+17%	15
dynamic	answer maintenance	-0.033	+59%	15
dynamic	automatic question resolution	-0.237	+79%	15

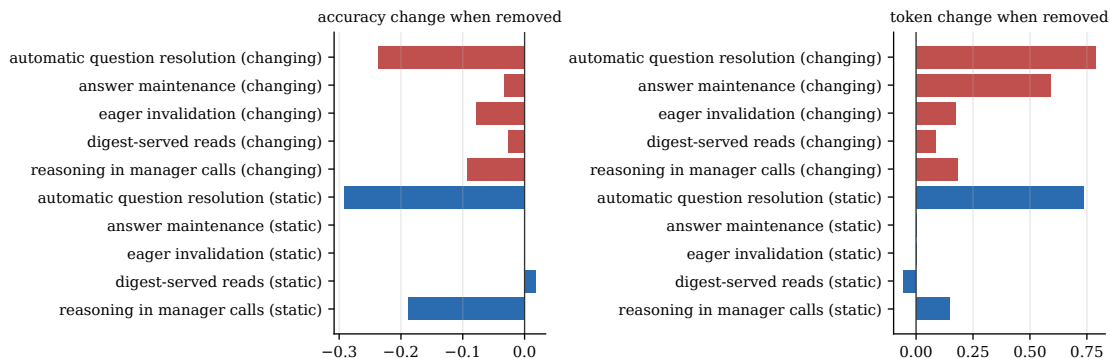


Figure 12: Ablations: change in accuracy (left) and in tokens (right) when one component of BACM is removed (paired held-out runs, 15 per cell; blue: static, red: changing repositories). Bars to the left in the accuracy panel and to the right in the token panel indicate that the component helps.

8.5 RQ4: Robustness

Reasoning control. Letting every agent reason more (effort “low” instead of “minimal”) raises independent agents to 0.93 on static and 0.63 on changing repositories, at 353k and 387k tokens per run. On static repositories this baseline is as accurate as BACM (difference -0.04 , $p_{\text{Holm}} = 0.26$), and BACM uses 23% more tokens than it (not significant, $p_{\text{Holm}} = 0.085$). On changing repositories BACM is more accurate by $+0.25$ ($p_{\text{Holm}} = 0.0006$) and again uses 13% more tokens (not significant, $p_{\text{Holm}} = 0.26$). Reasoning helps an agent read; it does not tell the agent that what it read has since changed.

Upper bound. With a symbol-index extractor, the same architecture is both the most accurate and the cheapest system in both environments (Table 5). The gap between BACM and BACM-Index is therefore the price of model-based extraction, and it is concentrated in the counting questions (Fig. 10).

Second model family. With Gemini 3.5 Flash-Lite (10 runs per cell, two held-out seeds; Table 9) the pattern on changing repositories repeats: accuracy 0.57 against 0.45 for independent agents, with 8 stale answers out of 62 against 37. On static repositories accuracy was similar (0.63 against 0.65) at 128k instead of 228k tokens. With so few runs we report these numbers without a test.

Table 9: Replication with a second model family (Gemini 3.5 Flash-Lite, free tier; held-out seeds that fit the free quota).

Architecture	runs	Acc. static	Tokens (k)	Acc. dynamic	Tokens (k)
Independent + evict	10	0.65	228	0.45	238
Selective memory (transparent)	10	0.63	128	0.48	157
BACM	10	0.63	128	0.57	162

8.6 Summary

Table 10 collects the answers to the four research questions.

Table 10: Answers to the research questions (held-out runs, GPT-5 mini unless noted).

Question	Answer
RQ1 accuracy, cost	Accuracy improves significantly on static and changing repositories; the average token saving is not significant and depends on the repository (Table 6 and Fig. 8).
RQ2 staleness	After a release, BACM leaves far fewer stale answers than every baseline; agents that reason more are stale most often (Fig. 11).
RQ3 mechanism	Rule-based question resolution matters most, followed by reasoning in the manager’s extraction calls; invalidation and maintenance matter only under change (Table 8).
RQ4 robustness	Agents that reason more match BACM on static repositories but not under change; the direction replicates with Gemini on few runs (Table 9).

9 Discussion

Reasoning quality is not state consistency. The most general observation of the evaluation is that two properties which are easily conflated are independent. Agents that reason more read

the repository more accurately, and on static repositories they reach the accuracy of BACM; but after a release they are stale for nearly every changed answer, because nothing in an agent’s context tells it that what it read earlier no longer holds. State consistency is a property of the memory, not of the reasoning applied to it, and it has to be provided by mechanism, here provenance, versions and dependency-aware invalidation, rather than by a stronger model.

Why accuracy improves. The gains come from two distinct mechanisms, and the experiments separate them. On static repositories, BACM and the selective memory produce identical runs, so the static gain over independent agents is entirely due to *sharing through a transparent interface*: facts are extracted once from focused views instead of being re-read by every agent, and questions are resolved by typed rules instead of being derived by an agent at minimal reasoning effort. The ablation of rule-based resolution confirms this (Table 8). Under change, BACM adds a second gain over the selective memory, which is due to *staleness tracking*: provenance and dependencies tell the manager exactly which facts a release invalidated, and demand-driven recomputation restores the ones that matter. The reasoning control shows that neither mechanism is a substitute for reasoning, nor reasoning a substitute for them: agents that reason more read more accurately, but they have no way to learn that what they read has changed.

What failed, and why it matters. Two intermediate designs did not work, and both failures are instructive. (i) Exposing memory as tools made the system *more* expensive than independent agents, because every memory operation is a full-context turn (Eq. (3)). Table 11 shows the effect on an earlier version of the benchmark: the tool interface more than doubled the number of model calls and the tokens relative to independent agents with eviction. The remedy is an interface, not a better data structure.

Table 11: First design: memory exposed to the model as tools (`kb_query`/`kb_publish`), on an earlier version of the benchmark (GPT-5 mini, 15 runs per cell, `httpx`, requests and flask $\times$ five seeds). Each memory operation is a full-context turn, so the tool interface multiplies turns and tokens.

Env.	Architecture	LLM calls	Tokens (k)	Accuracy
static	Independent	43	476	0.80
static	Independent + evict	59	217	0.88
static	Selective memory (tools)	112	332	0.93
static	BACM v1 (memory as tools)	137	441	0.89
dynamic	Independent	46	543	0.49
dynamic	Independent + evict	65	261	0.57
dynamic	Selective memory (tools)	114	339	0.57
dynamic	BACM v1 (memory as tools)	177	647	0.64

(ii) With model-based extraction, shared facts propagate the extractor’s errors to every agent. In our development runs, a single model call that counted the methods of every class in a file was wrong for a third of the classes; extracting each count from one class body, on demand and with a little reasoning in that small context, removed most of these errors at negligible cost. Spending reasoning where contexts are small, and keeping the agents’ contexts small, is the common thread of the final design.

Correctness of the memory itself. Both post-registration defects (Section 8.3) were of the same nature: the manager believed a fact to be valid, or gave up on rebuilding it, when the dependency information was incomplete. Such defects do not show up on static repositories at all, and they are invisible in aggregate accuracy unless stale answers are analysed separately. We consider this the main methodological lesson for work on shared agent memory: evaluations should report staleness

explicitly, under real changes, and the provenance of each fact must cover everything its value depends on.

Implications for practitioners. Three lessons transfer beyond code. First, make shared memory invisible to the model: publish from the tool layer and answer from rules, rather than adding memory tools. Second, give every shared fact provenance that covers everything its value depends on; a span that is too narrow silently disables invalidation, as our own defect showed. Third, the token saving of sharing depends on how much redundant reading the task induces; where agents read few small files, the bookkeeping can cost more than it saves, and a deployment should measure this before relying on it.

Limitations. The benchmark is narrow in four ways: it has five repositories in one language, one release jump per repository, and questions about the structure of the code (base classes, defining files, method counts) with exact answers. It therefore tests consistency of shared facts under change, not end-to-end software engineering such as producing a patch that passes the tests; whether the gains carry over to such tasks is untested. Facts enter memory only from files that an agent read; the manager never scans the repository on its own, except to re-read, with provenance, facts it already holds after a change. The results are for two model families at their least expensive reasoning settings, and the second family has only ten runs per cell, which supports the direction of the effect but not a claim of generality across model families. Each repository contributes one release jump. The ablations were run on the pre-registered code, before the two corrections.

Threats to validity. *Internal validity:* API responses are not fully deterministic; we freeze them in a cache and report the variation over held-out seeds. The two corrections were found by inspecting held-out results; we therefore report the frozen numbers next to the corrected ones and changed nothing else. *Construct validity:* the symbol-index configuration presumes a reliable extractor, which exists for code (language servers, tree-sitter) but not necessarily for other domains; we therefore report model-based extraction as the main result and the index only as an upper bound. *External validity:* five repositories with one release jump each, in one language; other languages and change patterns may behave differently.

Broader impact. Shared memory reduces redundant computation and therefore energy use, and staleness tracking reduces the risk that agents act on outdated information. Conversely, a shared memory concentrates errors: a single wrong fact reaches every agent. Systems built on this design should keep provenance visible to users so that answers can be traced to the code that supports them.

10 Conclusion and Future Work

We studied whether a shared context manager can keep the answers of parallel LLM agents correct on a changing codebase without making them more expensive. Our results indicate that correctness under change requires two conditions. Memory must be *transparent* to the model, so that sharing never costs additional full-context turns, and every shared fact must carry provenance and dependencies that cover everything its value depends on, so that a source change invalidates and recomputes exactly what it affects. Under these conditions, and with focused extraction that keeps model-extracted facts accurate, BACM answered 89% and 88% of held-out questions correctly on static and changing repositories, against 58% and 49% for independent agents with eviction, and cut stale answers after a release from 61 to 7 of 186. The accuracy gains are significant. Token efficiency, by contrast, is not established: the pooled changes (−22% and −26%) are not significant and depend on the workload. The main conclusion is therefore that BACM makes shared memory

consistent under change, not that it makes it cheaper. Agents that reason more match BACM’s accuracy on static repositories at lower cost, but not once the code changes, because reasoning quality is not state consistency. Counts extracted by the model remain the main source of error.

Future work. Four directions follow directly from the results. (i) *Wider evaluation.* The evidence should be extended to more repositories and release jumps, to other languages, to more seeds for the second model family, and above all to end-to-end coding tasks scored by tests, which we did not run. (ii) *Better extraction at the source.* Because shared errors are correlated, verifying counting facts by a second focused read, or by a lightweight parser where one exists, should close much of the gap to the symbol-index upper bound. (iii) *Adaptive sharing.* The token saving depends on how much redundant reading a task induces, so the manager could estimate when its bookkeeping pays off and otherwise fall back to independent operation. (iv) *Beyond code.* The formal model applies wherever facts have locatable sources, such as documents, tables or web pages, with spans defined by the structure of the source; evaluating BACM on end-to-end tasks in these settings is the natural next step.

A Reproducibility and Pre-registration

Code and data. The repository contains the system (`bacm/`), the benchmark (`bacm/env/realrepo.py`, `bacm/env/pyfacts.py`), the experiment suites, the analysis scripts and this paper’s asset generator (`paper/make_assets.py`); every number, table and figure in the paper is produced by that script from the run logs. All model responses are cached (`.llm_cache/`, first-writer-wins so parallel runs replay exactly), so every analysis can be re-derived without new API calls.

Determinism. The harness iterates over every set in sorted order, so a run does not depend on Python’s hash seed (tested). API responses are not perfectly deterministic, which is why we report confidence intervals over held-out seeds and freeze all responses in the cache.

Pre-registration. `paper/PREREGISTRATION.md` fixes the hypotheses, systems, held-out seeds and analysis, and records a SHA-256 fingerprint over the system’s source files. Every later change is an amendment with date, reason and new fingerprint: one harness-only change, one tool-layer fix for malformed model calls (affected runs re-run), dropping two baselines to fit the budget before any held-out result was inspected, and the two post-hoc corrections of Section 8.3. The pre-registered hypotheses were: H1, higher accuracy than independent agents with eviction (supported, static and dynamic); H2, fewer tokens than that baseline on changing repositories (not supported: the reduction is not significant); H3, higher accuracy than the selective memory on changing repositories (supported); H4, the comparison with the low-effort control, without a predicted direction.

Commands.

```
python -m bacm run --suite repobench2 \
  --seed0 200 --seeds 6 \
  --archs independent_evict,selective_llm,\
    bacm_llm,bacm_auto \
  --backend openai_compat --provider openai \
  --model gpt-5-mini --max-usd <cap> \
  --out results_real/final
# corrected re-runs (same seeds)
python -m bacm run ... \
  --archs bacm_llm,bacm_auto \
  --out results_real/final_fix2
```

```
python paper/replay_kinds.py # from cache
python paper/make_assets.py  # all assets
```

B Ground-truth validation

Before any model run we verified, on all five repositories: (i) the tool layer’s line-level parser returns the `ast` ground truth for every base fact of every question subject; (ii) every derived answer equals its rule applied to its parents’ ground-truth values; (iii) the class-body slicer used for focused extraction yields the same method count as the full file for every class (380 classes). The validation found and fixed two defects in our own ground truth: classes whose base shares the class’s name (an external class they shadow) were double-counted, and the counting definition was ambiguous about method aliases (`alias = method`); both fixes apply to every architecture.

References

- [1] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.
- [2] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [3] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.
- [4] Joseph Fiorese, Parth Parag Kulkarni, Ashmal Vayani, Song Wang, and Mubarak Shah. Learning to share: Selective memory for efficient parallel agentic systems. *arXiv preprint arXiv:2602.05965*, 2026.
- [5] Phillip I. Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 3 edition, 2005.
- [6] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007.
- [7] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. Maintaining views incrementally. In *ACM SIGMOD International Conference on Management of Data*, pages 157–166, 1993.
- [8] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable, demand-driven incremental computation. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 156–166, 2014.
- [9] Shanshan Han, Qifan Zhang, Yuhang Yao, Weizhao Jin, Zhaozhuo Xu, and Chaoyang He. LLM multi-agent systems: Challenges and open problems. *arXiv preprint arXiv:2402.03578*, 2024.
- [10] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

- [11] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025.
- [13] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [16] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [19] Yuzhen Mao, Jerry Gu, Aadi Chauhan, Qizheng Zhang, Hangoo Kang, and Azalia Mirhoseini. Decentralized multi-agent systems with shared context. *arXiv preprint arXiv:2606.10662*, 2026.
- [20] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):79:1–79:29, 2018.
- [21] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [22] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [23] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

- [24] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [26] Zhongming Yu, Naicheng Yu, Hejia Zhang, Wentao Ni, Mingrui Yin, Jiaying Yang, Yujie Zhao, and Jishen Zhao. Multi-agent memory from a computer architecture perspective: Visions and challenges ahead. *arXiv preprint arXiv:2603.10062*, 2026.
- [27] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [28] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [29] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.